

// I've intentionally left out some comments I would normally include,
 // so as to get you thinking about why this code does what it does.
 // I've dropped in some questions for thought as well.

int LED = 12;
 int BEEPER = 9;

// Why is this function necessary?
void establishContact() {

while (Serial.available() <= 0) {
 Serial.print('A'); // Why a capital A?
 delay(300);
 }
}

void setup() {
 // start serial port at 9600 bps:
 Serial.begin(9600);
 pinMode(A0, INPUT);
 pinMode(A1, INPUT);
 pinMode(LED, OUTPUT);
 pinMode(BEEPER, OUTPUT);
 establishContact();
}

void loop() {
 // Why bother with Serial.available()? Why not just Serial.read()?
if (Serial.available() > 0) {
 int inByte = Serial.read();
 analogWrite(LED, inByte);
 tone(BEEPER, inByte*3);
 }
}

// Why wouldn't Serial be available for writing?
if (Serial.availableForWrite()) {
 // Why divide by 4?
int firstSensor = analogRead(A0) / 4;

 // Delay 10ms to let the ADC recover.
 delay(10);

int secondSensor = analogRead(A1) / 4;

 Serial.write(firstSensor);
 Serial.write(secondSensor);
}

how many bytes
 are waiting
 in incoming
 queue?

outgoing queue
 AAAAA

queue of bytes sent
 (64 bytes)

Someone left
 me a message!

1 byte:
 0 ↔ 255

I've intentionally left out some comments I would normally include,
 # so as to get you thinking about why this code does what it does.
 # I've dropped in some questions for thought as well.

import serial
import pynput
import tkinter as tk

Why is this function necessary?
def establishContact(ser):

while True:
 # On the Arduino side, Serial.read checks Serial.available() first.
 # The Python code doesn't. Why?
 receivedByte = ser.read(1)

 # What's with all the "b" prefixes?
 if str(receivedByte) == "b'A":
 ser.reset_input_buffer()
 ser.write(b'A')
 return

def main():
 ser = serial.Serial('/dev/ttyUSB0', 9600)
 establishContact(ser)

Window and mouse, just so I have something to read
 root_window = tk.Tk()
 mouse = pynput.mouse.Controller()

while True:
 root_window.update() # Refresh the window so I can see mouse

 potValues = ser.read(2)
print("Pot values are", potValues[0], potValues[1])

 # Why does this check below accomplish?
if ser.out_waiting <= 60:

 # Why divide by 12?
 ledValue = **int**(sum(mouse.position) / 12)

 # What's up with the conversion below?
 ledValueByteArray = (ledValue).to_bytes(1, byteorder='big')
 ser.write(ledValueByteArray)

if __name__ == "__main__":
 main()